1. Risk Analysis

In this step, we identify possible risks that could affect the development or operation of the Coffee POS System. These risks may come from technical issues, security problems, user experience challenges, or project management delays.

Example risks identified for the project include:

- Changes to system features requested late in development
- Errors in order processing or menu updates
- Unauthorized access to the business dashboard
- System crashes when many orders are processed
- Business owners finding the system difficult to use
- Team members falling behind schedule

1.1. **Risk Assessment and Management**

After identifying the risks, each risk is evaluated based on two factors:

A. Probability – How likely the risk is to occur
B. Impact – How serious the consequences would be if it occurs

The risks are then categorized and prioritized so the team can focus on the most important ones.

Risk Assessment Table:

Risk ID	Risk Description	Category	Probability	Impact
R1	Major feature changes requested late in development	Scope	Medium	High
R2	Errors when saving or updating menu items	Technical	Medium	High
R3	Unauthorized users accessing the business dashboard	Security	Low	Very High
R4	Business owners finding the system difficult to use	Usability	Medium	Medium
R5	System slowdown or crash during high order activity	Technical	Low	High
R6	Delays in development tasks by team members	Resource	Medium	Medium

From the risk assessment table, several risks were identified as higher priority due to their potential impact on the system. R1 (major feature changes requested late in development) and R2 (errors when saving or updating menu items) both have medium probability and high impact, meaning they could significantly affect the project schedule and the functionality of the Coffee POS System if not properly managed. These risks will be monitored closely during development and addressed early in the sprint planning process.

R3 (unauthorized access to the business dashboard) has a low probability but very high impact, making it a critical security risk. Even though it is less likely to occur, a security breach could compromise sensitive business data and user accounts. Therefore, strong authentication and access control measures must be implemented to reduce this risk.

Overall, the risk assessment allows the team to prioritize high-impact risks, monitor medium risks during development, and implement preventive actions to minimize their effects on the project timeline and system performance.

1.2. **Mitigation / Preventive Actions**

After assessing the identified risks, mitigation strategies were developed to reduce the likelihood of the risks occurring and to minimize their impact if they happen during the

development of the Coffee POS System. These strategies focus on preventing technical issues, improving security, ensuring usability, and maintaining steady project progress.

Risk ID	Mitigation Strategy
R1	Finalize main features early and review new requests before adding them
R2	Test menu creation and editing features thoroughly before deployment
R3	Implement secure login, password encryption, and role-based access control
R4	Design a simple interface and conduct user testing with sample users
R5	Optimize database queries and test the system with large order volumes
R6	Assign clear responsibilities and monitor progress through weekly sprint meetings

Overall, these mitigation strategies help reduce the overall project risk by addressing technical, security, usability, and project management challenges before they become serious issues. By planning these preventive actions in advance, the development team can improve the stability, reliability, and success of the Coffee POS System project.

1.3. **Risk Matrix**

A risk matrix is used to visually evaluate the level of risk by comparing the probability of occurrence with the impact on the project. This method helps the team quickly identify which risks require the most attention and prioritize appropriate responses.

In this project, the identified risks were placed into the matrix according to their assessed probability (Low, Medium, High) and impact (Medium, High, Very High). Risks that have both higher probability and higher impact are considered more critical and must be monitored closely during development.

Impact ↓ / Probability →	Low Probability	Medium Probability	High Probability
Very High Impact		R3	
High Impact	R5	R1, R2	
Medium Impact		R4, R6	
Low Impact			

From the matrix, we can understand that R1 (late feature changes) and R2 (menu update errors) fall into the medium-probability and high-impact category, making them important risks to monitor during development. R3 (unauthorized access) has a very high impact even though its probability is low, meaning strong security controls must still be implemented.

Other risks such as R4 (usability challenges) and R6 (team delays) fall into the medium category and should be monitored throughout the project to ensure they do not affect the development timeline or user experience. R5 (system slowdown during heavy order activity) has a lower probability but still a high impact, so performance testing and system optimization will be necessary.

Overall, the risk matrix helps the development team prioritize high-impact risks and focus mitigation efforts where they are most needed, ensuring the Coffee POS System can be developed and deployed with minimal disruption.

2. Code Security and Data Privacy

We used a defense in depth strategy and implemented multiple architectures and measures to ensure the security of user operations and the privacy user data.

1. Code-Level Security Mechanisms

- SQL Injection Prevention: The system utilizes JDBC Prepared Statements for all database interactions. By using parameterized queries, the application ensures that user input is never executed as code, neutralizing one of the most common threats to relational databases.

- Strong Typing and Memory Safety: By leveraging Java SE 21, the project benefits from a managed memory environment and strict type-checking. This prevents buffer overflow attacks and class-casting errors that are common in lower-level languages.

- Input Validation: UI components within LoginFrame.java and ManagmentFrame.java include logic to sanitize and validate user inputs before they are processed by the business logic layer or persisted to MySQL.

2. Data Privacy and Encryption

- Credential Protection (BCrypt): Passwords are never stored in plain text. The system implements BCrypt password hashing, which incorporates a unique salt for every user. This ensures that even in the event of a database breach, user passwords remain computationally infeasible to crack.

- Sensitive Data Masking: Within the JavaX Swing interface, sensitive fields (such as password entries) utilize JPasswordField, ensuring that credentials are not visible on-screen during entry and are handled as char[] arrays rather than Strings to minimize their footprint in memory.

- Encryption at Rest: For high-sensitivity columns in the MySQL database (e.g., employee personal info), the system supports Transparent Data Encryption (TDE) or application-level encryption modules to ensure data remains unreadable without the proper decryption keys.

3. Application-Level Security

3.1 Authentication (AuthN)

The LoginFrame serves as the primary gatekeeper. The authentication flow is as follows:

- **Identity Matching:** The system queries the users table for a unique username match.
- **Hash Verification:** The entered password is compared against the stored hash using the BCrypt.checkpw() method.
- **Session Initiation:** Upon success, a local session object is created to track the user's identity throughout the application lifecycle.

3.2 Authorization (AuthZ)

- The application implements a Role-Based Access Control (RBAC) model to enforce the Principle of Least Privilege. Swing UI components (buttons, menus) are dynamically enabled or hidden based on these roles. For example, the Admin button is programmatically disabled for users with Staff clearance.

4. Auditing and Traceability

- Audit Logs: Every significant transaction (e.g., deleting an order, changing a price) is recorded in a dedicated audit_log table in MySQL.

- Timestamping: All records include automated created_at metadata to provide a chronological history of system activity.

- User Attribution: Every log entry is tied to a specific user_id, allowing administrators to trace actions back to the responsible individual.

5. Secure Infrastructure and Deployment

- Environment Secret Management: Database credentials and system keys are stored in external properties, ensuring sensitive information is never hardcoded in the source files.
- Dependency Auditing: The project’s external libraries, such as MigLayout and the MySQL Connector/J, are regularly audited for known vulnerabilities to maintain a secure supply chain.
- CI/CD Security: GitHub Actions workflows are configured to run automated tests and static analysis, ensuring that no insecure code patterns are merged into the main branch.

3. Implementation Roadmap

Phase 1: Environment Setup & Architecture Review (Week 1)

- **IDE & Dependency Configuration:** Set up the development environment (Eclipse) and ensure the MigLayout library (miglayout15-swing.jar, etc.) is properly linked in the build path.
- **Codebase Familiarization:** Review the existing classes (POS.java, Menu.java, ManagmentFrame.java).
- **UML Alignment:** Cross-reference the provided UML diagram with the actual implementation to identify any deviations or missing components.
- **Version Control Initialization:** Ensure the .gitignore is optimized for Java projects and push the baseline code to a repository (like GitHub).

Phase 2: Refactoring & Database Integration (Week 2)

- **Decoupling UI and Logic:** Separate the Swing GUI components (like LoginFrame.java) from the underlying business logic.
- **Database Setup:** Replace any in-memory or hardcoded arrays for Drink and Dessert with a relational database (e.g. MySQL).
- **Data Access Object (DAO) Layer:** Create DAO classes to handle CRUD (Create, Read, Update, Delete) operations for inventory items, transactions, and employee records.

Phase 3: Security & Access Control (Weeks 4-5)

- **Authentication Hardening:** Use secure password hashing algorithms (such as BCrypt).
- **Role-Based Access Control (RBAC):** Distinctly separate permissions for the Admin class versus standard cashier roles. Ensure that only administrators can access ManagmentFrame.java to alter prices, view aggregate sales or edit the products in the menu.

Phase 4: Business Intelligence & Advanced Analytics (Weeks 6-7)

- **Data Collection Pipeline:** Implement logging for every transaction, capturing timestamps, item combinations, and total amounts.
- **Statistical Dashboards:** Create a new UI panel for administrators that visualizes daily, weekly, and monthly sales statistics.

- **Predictive Features:** Introduce a lightweight machine learning or statistical model that analyzes historical sales data to predict future inventory needs (e.g., forecasting how many espresso beans will be needed for the upcoming weekend and expecting the upcoming profits).

Phase 5: Testing, CI/CD, & Documentation (Week 8)

- **Unit & Integration Testing:** Write JUnit tests for the core business logic, ensuring calculations (totals, discounts) are flawlessly accurate.
- **Continuous Integration:** Set up a simple CI pipeline (e.g., GitHub Actions) to automatically compile the Java code and run tests on every commit.
- **Technical Reporting:** Create documentation, including an updated UML diagram, a database schema map, and a formal project report detailing the architectural decisions and algorithms used.

Phase 6: Deployment & Future Exploration

Executable Packaging: Bundle the Java application and its dependencies (MigLayout) into a single, runnable .jar file or a native installer using tools like jpackage.

4. Validation and Testing Plan

1. Testing Methodology
    - Unit Testing: Individual components such as the BCrypt hashing mechanism and the updateTotal() function in the POS class will be tested to ensure mathematical and cryptographic accuracy.
    - Integration Testing: We will verify the seamless interaction between the LoginFrame, the Authorization Engine, and the ManagementFrame to ensure data flows correctly across the application layer.
    - System Testing: The complete end-to-end "Order-to-Receipt" workflow will be tested to ensure the entire system functions as a unified unit under simulated café operational conditions.
2. Validation Methods
    - Role-Based Access Validation: Using the five-level clearance system (L0-L4), we will validate that users can only access UI components permitted by their specific role (e.g., ensuring Staff cannot see the Management Panel).

- UI/UX Validation: We will validate that the JavaX Swing interface, managed by MigLayout, remains stable and organized at the minimum required resolution of 1024x768.
- Security Validation: Testing the Secure Credential Validation process to confirm that sensitive data masking is active and that unauthorized access attempts are blocked by the system.

3. Performance Evaluation
    - Processing Speed: The system will be evaluated on its ability to handle "Rapid Order Processing" to ensure it meets the high-traffic demands of a professional café environment.
    - Response Latency: Evaluation of the Authorization Engine to ensure that permission checks do not introduce perceptible delays when opening role-specific screens.
    - Resource Efficiency: Monitoring the Java SE 21 environment to ensure the application maintains a low memory footprint during peak usage.

5. Success Criteria

The project will be deemed successful if it meets the following benchmarks:

1. Security Integrity: 100% of user passwords must be encrypted using BCrypt hashing, with no plain-text credentials stored in the system.
2. Functional Accuracy: The system must successfully execute all core features, including order processing, receipt generation, and real-time sales analytics, with zero critical bugs.
3. Access Control Compliance: The "Principle of Least Privilege" must be fully operational, with the UI dynamically hiding or disabling features based on the user's clearance level (L0-L4).
4. Operational Reliability: The interface must provide responsive feedback via JOptionPane and maintain a stable layout across the specified without graphical artifacts.